

Wave 1 Validation Matrix

Denai Clinical Platform — Deterministic Extraction Validation Governance

Classification: Extraction Validation Governance — Deterministic Preservation **Depends**

on: Synchronization Analysis, Render Trigger Chain Analysis, Architecture Risk

Consolidation, DOM Lifecycle Audit, Wave 1 Execution Playbook, Wave 1 Candidate Registry

Scope: All Wave 1 extraction tiers (Tier 1 through Tier 5). Every validation dimension applies per-extraction unless explicitly scoped to a specific tier.

Document Purpose

This matrix defines the complete, deterministic validation and regression verification protocol required after every individual Wave 1 extraction. It is not a test plan. It is a governance contract. Every dimension defined here encodes a specific documented risk from the prior analyses. No dimension may be waived. No dimension is aspirational — all are required before the next extraction begins.

The matrix is organized into twenty validation dimensions. Each dimension defines: what must be tested, what must remain identical to the Tier 0 behavioral baseline, what signals extraction failure, what requires immediate rollback, and what constitutes behavioral drift, orchestration drift, or timing drift.

Tier 0 Behavioral Baseline — Pre-Condition for All Validation

Before any extraction begins, the Tier 0 baseline must be established and frozen. All validation in this matrix is relative to this baseline. A baseline that has not been formally established renders all downstream validation invalid.

Baseline Composition

The Tier 0 baseline consists of:

R-series output snapshots for all twelve regression cases defined in Playbook Section 10.1 (R01–R12). Each snapshot records the exact DOM content, localStorage state, and console

output produced by the application for that case.

T-series timing measurements for all five timing cases defined in Playbook Section 10.2 (T01–T05). Each measurement records the observed duration with timestamp precision to $\pm 5\text{ms}$.

Module-scope variable inventory at application startup: a complete list of all module-scoped variables, their initial values, and their types. This list is the reference for the no-new-import validation dimension.

Timer registry snapshot at the point immediately after a single render cycle completes: a record of all pending timer IDs, their registered durations, and the DOM elements they reference.

Observer registry snapshot: a record of all active IntersectionObserver instances, their target elements, and the values captured in their closures at creation time.

localStorage state snapshot for a known-good patient set: exact serialized values for all patient records, active patient ID, and history entries.

Import dependency graph: a static trace of all import relationships present in the codebase before Wave 1 begins. This graph is the reference for the dependency validation dimension.

The Tier 0 baseline is immutable. It is tagged `wave1-baseline` in version control and must not be modified under any circumstance during Wave 1 activity.

Dimension 1: Required Validation Scope

Per-extraction minimum scope

Every extraction, regardless of tier, must be validated against the following minimum scope before the next extraction begins:

- The extracted module executes in isolation in a pure Node.js environment (`node module.js`) with no application context, no DOM, no browser APIs, no `s` , and no `tempState` . If the module throws, imports fail, or requires mocking to execute, the extraction fails isolation validation immediately.
- All call sites in the main application file that previously referenced the extracted function now reference the imported version. No duplicate inline definition of the extracted function remains.
- The main application's behavior for all twelve R-series regression cases is byte-for-byte

identical to the Tier 0 baseline.

- No new module-scoped variable has been introduced in the main application file as a result of the extraction.
- No existing module-scoped variable has been removed or renamed in the main application file as a result of the extraction.

Tier-specific scope additions

Tier 1 (constants and lookup tables): Validation scope is limited to import resolution and isolation. No timing validation is required. No DOM state validation is required. R-series cases R01–R06 must be re-verified to confirm label and classification outputs are unchanged.

Tier 2 (formatters): Validation scope extends to output equivalence for all formatter inputs present in R-series cases. Formatted strings must be byte-for-byte identical to Tier 0 baseline strings. No timing validation is required unless a formatter is called from within a timer callback.

Tier 3 (validators): Validation scope extends to setState call site behavior. Any change in which fields are accepted or rejected by StateValidator is a regression. T04 (state autosave timing) must be re-verified.

Tier 4 (clinical calculation helpers): Full validation scope applies, including all T-series timing cases, all DOM state cases, both calcAI and ClinicalEngine output paths, and the two-pass SVG compound highlight sequence. This is the highest-risk tier. No Tier 4 extraction proceeds without completing 100% of all twenty validation dimensions.

Tier 5 (conditional extractions): Same as Tier 4 scope for any extraction that touches a function called from within a render path. For showToast() specifically: DOM mutation behavior, caller context verification, and absence of S-reads must all be confirmed.

Dimension 2: Required Behavioral Baselines

Baseline coverage requirements

The behavioral baseline for each extraction candidate must cover a minimum of three input categories:

Category A — Normal clinical case: A fully populated patient state with all required clinical fields present, a valid tooth number, a non-restorative condition, and a selected treatment. This case exercises the primary render path.

Category B — Edge case or missing-data case: A patient state with one or more clinical fields at boundary values (minimum, maximum, or null/undefined where applicable). This case exercises defensive branches and early-exit paths.

Category C — Multi-site or multi-tooth case (where applicable): A patient state with `S.multiSite === true` and `S.site2Tooth` populated. This case exercises the compound branch and the two-pass SVG path. Required for all Tier 4 candidates. Required for any Tier 1 or Tier 2 candidate whose output differs between single-site and multi-site modes.

Baseline recording format

For each input category, the baseline record must contain:

- The exact input arguments passed to the function (serialized as JSON where possible)
- The exact return value of the function (serialized as JSON where possible)
- The function's execution context (which caller invoked it, synchronous or async)
- Any module-scope variables that were read during execution (must be none for Wave 1 candidates — if any are found, the candidate fails purity and is reclassified as forbidden)

Equivalence standard

Post-extraction output must be byte-for-byte identical to the recorded baseline for all Category A, B, and C inputs. The only permitted difference between pre- and post-extraction execution is the module boundary — the function's source location changes; its observable output does not.

Any difference in output, including whitespace, numeric precision, string case, or ordering of object keys, constitutes a behavioral regression and requires immediate rollback.

Dimension 3: Required Render Invariants

The following render behaviors must remain identical to the Tier 0 baseline after every extraction. Any deviation is a render regression.

AI card render invariants

- `buildAICardStructure()` must execute a full `innerHTML` replacement of `#aiCardBody` when and only when `body.dataset.built` is absent or not equal to `'1'`.
- `updateAICard()` and `updateAICardMulti()` must execute patch operations on `#aiCardBody` child elements when and only when `body.dataset.built === '1'`.

- The `dataset.built` flag must be set to `'1'` by `buildAICardStructure()` on successful build. It must not be set by any other function, including any extracted Wave 1 module.
- The `dataset.built` flag must be cleared (implicitly, via `innerHTML` overwrite) by `showSkeleton('aiCardBody')` and by the `withErrorBoundary` catch block. No extracted Wave 1 module may alter when or whether these clearing events occur.
- After a `withErrorBoundary` catch on `#aiCardBody`, subsequent calls to `updateAICard()` must be silent no-ops against the error fallback markup. This terminal error state must persist indefinitely until the next `switchPatient()`. No extracted Wave 1 module may introduce recovery behavior.

Comparison table render invariants

- `renderComparisonTable()` must produce a full `innerHTML` replacement of `#comparisonTableBody` on every invocation. No patch path may be introduced.
- `lazyRenderComparisonTable()` must gate synchronous render on element visibility at call time. The observer-deferred path must remain available when the element is offscreen.
- `_compTableObserver` must capture `state` and `ai` by closure at creation time. The captured values must not be refreshed by any Wave 1 extraction. Stale closure behavior at scroll-into-view time is a documented existing contract.
- `toggleMultiSite()` must continue to execute a direct `innerHTML = ''` pre-wipe of `#comparisonTableBody` before the render sequence. No extraction may interpose between the pre-wipe and the render.

Material render invariants

- `renderMaterial()` must defer all `innerHTML` writes to `#matPrimary` and `#matAlt` by exactly 160ms via `_matFadeTimer`. No extraction may convert this deferred write to a synchronous write.
- The cancellation-via-overwrite pattern (`clearTimeout(_matFadeTimer)` at the top of `renderMaterial()`) must remain intact. The last caller must always win. No extraction may introduce a queuing mechanism.

Compound render invariants

- The two-pass SVG compound highlight sequence must be preserved: Pass 1 fires inside `renderMainPanels(effectiveState, ai)` via `updateToothHighlight(effectiveState.tooth)`. Pass 2 fires directly from `render()` via

`updateToothHighlight(primaryTooth)` and `updateToothHighlight2(secondaryTooth)`, overwriting Pass 1. Pass 2 is the authoritative result.

- No extraction may alter the sequencing of these two passes or introduce a mechanism by which Pass 1 survives as the final state.

Patient list render invariants

- `renderPatientList()` must execute a full `innerHTML` replacement of `#patientList` on every invocation. No patch path may be introduced.
 - The sidebar must continue to reflect persisted `localStorage` state, not runtime `s` state.
-

Dimension 4: Required Synchronization Invariants

The following synchronization contracts must survive every extraction unchanged.

render(S) orchestration sequence

The sequence `render(S) → ClinicalEngine.process(S)` or

`ClinicalEngine.processCompound(S) → renderMainPanels(effectiveState, ai) → individual panel renders → SVG Pass 2 overwrite` must execute in this exact order. No Wave 1 extraction may insert, remove, or reorder any step in this sequence.

switchPatient() mixed DOM state window

The 300ms mixed DOM state window produced by `switchPatient()` must remain intact.

The sequence is: `Object.assign(S, newPatient) → synchronous DOM resets`

`(showSkeleton, History.render, renderPatientList, buildEditForm) → 300ms deferred`

`setTimeout(() => { render(S); }, 300)`. This window — in which the sidebar and history show new patient data while main panels still show old patient data — is a frozen behavioral contract. No extraction may close this window, narrow it, or extend it.

saveState → renderPatientList side-effect chain

The chain `setState() → saveState() (200ms _saveStateTimer) →`

`renderPatientList()` must execute in this exact sequence with the 200ms debounce interval preserved. No extraction may introduce a direct call to `renderPatientList()` from any extracted module. No extraction may alter the debounce interval.

Preview vs. apply race condition

The documented race condition between `debouncedUpdatePreview()` (120ms CleanupRegistry timer) and `selectTx()` / `applyPreset()` (synchronous) must remain unresolved — this is an existing behavioral contract. No extraction may introduce coordination between these paths.

whatIfApply selective promotion boundary

The field set promoted from `tempState` to `S` by `whatIfApply ( bone , hygiene , occlusion , smoking , diabetes , remainingStructure , endodonticStatus , parafunction )` versus the fields intentionally excluded (`tooth , name , age , condition , tx , gender`) must remain exactly as defined. No extraction may alter which fields are in scope for promotion.

Dimension 5: Required Persistence Invariants

Transient flag blocklist

The destructure pattern `const { editing, whyOpen, historyOpen, ...serializable } = S` inside the `saveState()` timer callback defines which fields are excluded from `localStorage` persistence. This blocklist must survive every extraction unchanged. After any Tier 1 extraction involving `DEFAULT_STATE` field values, verify that `editing`, `whyOpen`, and `historyOpen` are still present in the main application file's `DEFAULT_STATE` object and are still excluded from the serializable spread.

Patient record integrity

After every extraction, perform a full `localStorage` inspection for all patients in the regression test set. Each patient record must contain exactly the fields present in the Tier 0 baseline snapshot — no new fields added, no existing fields removed, no field values altered.

History entry integrity

History entries are keyed by `S.id` and stored per-patient. After every extraction, verify that history entries for the regression test patients are identical to the Tier 0 baseline — correct patient association, correct label and detail text, correct timestamp format.

Active patient ID integrity

`getActivePatientId()` must return the same active patient ID as the Tier 0 baseline for all regression cases. No extraction may alter which patient is treated as active.

Save timing integrity

The 200ms `_saveStateTimer` debounce must produce a `localStorage` write at 200ms (± 20 ms) after the last `setState()` call. The 800ms `_notesDebounce` must produce a `S.notes` mutation at 800ms (± 50 ms) after the last notes input event. These timings are defined in T03 and T04 and must be re-verified after any Tier 3 or Tier 4 extraction.

Dimension 6: Required Timing Invariants

The following timing windows are documented behavioral contracts. Any measurable deviation after an extraction is a timing regression requiring immediate rollback.

T01 — Material render 160ms window

What to measure: Time from `renderMaterial()` invocation to the moment `#matPrimary` content updates in the DOM. **Required value:** 160ms $\pm$ 20ms. **Measurement method:** Browser performance profiler or manual stopwatch against a known-trigger render. Record three consecutive measurements and confirm all are within tolerance. **Regression trigger:** Any single measurement outside the 160ms $\pm$ 20ms window. Any extraction in Tier 4 that changes the observed window by any measurable amount, even if still within tolerance, must be flagged for investigation before proceeding. **What this validates:** `_matFadeTimer` has not been cancelled, replaced, or altered by the extraction. The 160ms deferred write pattern is intact.

T02 — Patient switch 300ms deferred render

What to measure: Time from `switchPatient()` invocation to the moment main panel DOM content updates (specifically `#aiCardBody` content change). **Required value:** 300ms $\pm$ 30ms. **Measurement method:** Browser performance profiler. The measurement window starts at the patient list click event and ends at the first observed `innerHTML` change on `#aiCardBody`. **Regression trigger:** Any measurement outside 300ms $\pm$ 30ms. **What this validates:** The anonymous `setTimeout(() => { render(S); }, 300)` in `switchPatient()` has not been altered. The deferred render window is intact.

T03 — Notes autosave 800ms debounce

What to measure: Time from the last keystroke in the notes textarea to the moment `S.notes` is mutated. **Required value:** 800ms $\pm$ 50ms. **Regression trigger:** Any measurement outside tolerance. **What this validates:** `_notesDebounce` has not been altered.

T04 — State autosave 200ms debounce

What to measure: Time from `setState()` invocation to the moment `renderPatientList()` fires (measurable by observing `#patientList` innerHTML change). **Required value:** 200ms $\pm$ 20ms. **Regression trigger:** Any measurement outside tolerance. **What this validates:** `_saveStateTimer` has not been altered. The `saveState() → renderPatientList()` side-effect chain is intact.

T05 — Typewriter animation 30ms interval

What to measure: Time between consecutive character appends to `recText` inside `#aiCardBody`. **Required value:** 30ms $\pm$ 5ms per interval. **Regression trigger:** Any interval measurement outside tolerance, or any observation that the typewriter animation no longer fires after a render. **What this validates:** `_typewriterTimer` (30ms `setInterval`) has not been altered. The cancellation-on-next-invocation pattern is intact.

Compound timing: T01 + T02 combined

After any Tier 4 extraction, verify the compound scenario: a patient switch followed immediately by a render with material change. The observed sequence must be: 300ms deferred render fires $\rightarrow$ `renderMaterial()` called $\rightarrow$ 160ms timer fires $\rightarrow$ `#matPrimary` updates. Total observed time from patient switch click to `#matPrimary` update: 460ms $\pm$ 50ms. Any deviation indicates the two timer chains have been coupled or decoupled by the extraction.

Dimension 7: Required DOM Invariants

The following DOM behaviors must remain identical to the Tier 0 baseline after every extraction.

innerHTML replacement map invariants

Every element listed in the innerHTML Replacement Map (DOM Lifecycle Audit, cross-system summary) must continue to be replaced via full `innerHTML` write on the same triggers as in the Tier 0 baseline. No extraction may introduce partial writes (patching) to any element that was previously subject to full replacement, and no extraction may introduce full replacement to any element that was previously patched.

Specifically:

- `#aiCardBody`: full `innerHTML` replacement only when `dataset.built` is absent. Patch-only when `dataset.built === '1'`. This distinction must be preserved exactly.

- `#comparisonTableBody`: full `innerHTML` replacement on every render. No patch path may be introduced.
- `#matPrimary` and `#matAlt`: full `innerHTML` replacement only inside the `_matFadeTimer` callback. No synchronous write may be introduced.
- `#patientList`: full `innerHTML` replacement on every `renderPatientList()` call.
- `#editFields`: full `innerHTML` replacement on every `buildEditForm()` call.
- `#historyList`: full `innerHTML` replacement on every `History.render()` call.

DOM element existence invariants

After every extraction, the following elements must exist in the DOM and must contain the same content as the Tier 0 baseline for the corresponding regression case: `#aiCardBody`, `#comparisonTableBody`, `#compoundSummaryBody`, `#patientList`, `#matPrimary`, `#matAlt`, `#editFields`, `#historyList`, `#previewModeBanner`, `.btn` (report button).

dataset.built flag invariants

`body.dataset.built` on `#aiCardBody` must be set to `'1'` after `buildAICardStructure()` completes successfully. It must be absent (or the element replaced) after `showSkeleton('aiCardBody')` or after a `withErrorBoundary` catch on `#aiCardBody`. No extracted module may set, clear, or read this flag. Verify flag state after every render cycle in the post-extraction DOM state validation (Playbook Section 8.5).

SVG attribute invariants

SVG tooth path elements must have the same `fill`, `stroke`, and `class` attributes as the Tier 0 baseline after a complete render cycle for the same input state. In compound mode, the second-pass attribute values must be the authoritative final state. First-pass values are intentionally overwritten and must not be preserved.

Preview banner invariants

`previewModeBanner.style.display` must be `'flex'` after `updatePreview()` completes and `'none'` after `whatIfApply` click handler fires. It must NOT be reset by `switchPatient()` — stale banner visibility on patient switch is the existing behavior and must be preserved. No extracted module may introduce a reset of this element on patient switch.

Dimension 8: Required localStorage Invariants

Structural invariants

The structure of every `localStorage` key must be identical to the Tier 0 baseline after extraction. This includes: key names, value serialization format, field presence, and field ordering within serialized objects (if ordering is deterministic).

Transient flag exclusion invariants

The fields `editing`, `whyOpen`, and `historyOpen` must never appear in serialized patient records in `localStorage`. After any Tier 1 extraction involving `DEFAULT_STATE`, inspect the serialized output of `saveState()` directly to confirm these fields are excluded. A single occurrence of any of these field names in a persisted record is a persistence regression.

Cross-patient contamination invariants

After executing regression case R12 (rapid patient switch, two switches within 300ms), inspect `localStorage` for all patient records and confirm that no patient record contains field values from a different patient. This is the save-after-switch corruption pattern (RF-08). While Wave 1 extractions do not touch the save path, cross-patient contamination in the test environment signals that the baseline environment is unsafe and all extraction work must halt.

History entry key invariants

History entries must be keyed by the exact patient ID that was active at the time `History.add()` was called. After regression case R12, confirm that history entries generated during the rapid switch sequence are attributed to the correct patient. A history entry attributed to the wrong patient is a History ID scope regression.

Dimension 9: Required Import/Export Validation

Export format invariants

All Wave 1 extracted modules must use named exports only. No default exports. After extraction, verify:

- The extracted module file contains at least one `export` keyword and zero `export default` statements.
- The main application file contains an `import { functionName } from`

'./path/to/module' statement for each extracted function.

- The import statement is placed in the designated Wave 1 imports section at the top of the main application file.

No-new-import invariants

After every extraction, the main application file's import section must contain no new imports beyond the Wave 1 module import being added. No npm package import may be added. No import from any file that contains render, setState, saveState, S, or tempState may be added to any extracted module.

No-barrel-file invariants

No `index.js` or barrel file may be created during Wave 1. Each extracted module is imported directly. After every extraction, confirm no `index.js` has been created in the `utils/` or `helpers/` directory.

Call site replacement completeness

After every extraction, the reverse dependency scan (Playbook Section 18.3) must be re-executed to confirm that every call site in the codebase references the imported version of the extracted function. A call site that still references an inline definition that has been deleted constitutes a broken call site. A call site that references an inline definition that was inadvertently left in place after extraction constitutes a duplicate definition violation.

Dimension 10: Required Dependency Validation

Direct dependency resolution

For every extracted module, all direct dependencies must resolve without importing from the main application file. After extraction, trace the full import chain of the extracted module. If any link in the chain imports from the main application file, the extraction is contaminated.

Depth-2 dependency scan

For every direct dependency of the extracted module, repeat the purity scans defined in Playbook Section 4.1 (S-read, S-write, DOM API, Browser API, Timer, Render call, Module-scope variable). Any depth-2 dependency that fails any scan contaminates the candidate and requires rollback.

Circular dependency prohibition

After every extraction, trace the complete import graph from the extracted module's file location. Confirm there is no cycle at any depth. A circular dependency at any depth is a dependency regression requiring immediate rollback.

npm package invariants

After every extraction, confirm that `package.json` has not changed. No new npm packages may be added during Wave 1. If the extracted module uses an npm package, confirm it was already in `package.json` before extraction and that it does not register globals, mutate prototypes, or register browser APIs on import.

Dimension 11: Required Rollback Triggers

The following events require immediate rollback to the most recent passing rollback checkpoint. No attempt to fix in place. Roll back first, understand the failure, then re-plan.

Automatic rollback triggers (no judgment required)

RB-01: Isolation failure. The extracted module fails to execute in a pure Node.js environment. Any import error, any runtime error, any reference to browser globals constitutes isolation failure.

RB-02: Behavioral output divergence. Any R-series regression case produces output that differs from the Tier 0 baseline in any respect, including whitespace, numeric precision, string case, key ordering, or field presence.

RB-03: Timing window deviation. Any T-series timing case falls outside its defined tolerance window after extraction (T01: 160ms $\pm$ 20ms, T02: 300ms $\pm$ 30ms, T03: 800ms $\pm$ 50ms, T04: 200ms $\pm$ 20ms, T05: 30ms $\pm$ 5ms).

RB-04: New module-scope variable. Any new module-scoped variable appears in the main application file after extraction that was not present in the Tier 0 module-scope variable inventory.

RB-05: New timer registration. The browser's task queue after extraction shows a timer or observer registration that was not present in the Tier 0 timer registry snapshot.

RB-06: dataset.built regression. Any observation that `#aiCardBody` undergoes a structural rebuild when it should patch, or undergoes a patch when it should rebuild, or that `dataset.built` is set by any function other than `buildAICardStructure()`.

RB-07: Comparison table content mismatch. The comparison table shows different row values than the Tier 0 baseline for the same input state, including mismatch caused by observer stale closure.

RB-08: S appears in extracted module scope. Any grep, lint, or manual review finds a reference to `s` (the global state object) in the extracted module file.

RB-09: Unexpected import required. The extraction requires importing something not listed in the pre-extraction call site inventory.

RB-10: Module-scoped variable discovered mid-extraction. A mutable module-scoped variable is discovered inside the candidate after extraction begins that was not visible from the function signature during static analysis.

RB-11: Cross-patient contamination detected. `localStorage` inspection at any point during Wave 1 validation reveals a patient record containing another patient's field values.

RB-12: calcAI / ClinicalEngine divergence altered. After any Tier 4 extraction, running R01 and R02 against both `calcAI(S)` and `ClinicalEngine.process(S)` produces a difference that did not exist in the Tier 0 baseline.

Judgment-required rollback triggers

RB-J1: Formatting output near-miss. A formatter produces output that appears equivalent but differs in a non-visible character (zero-width space, different Unicode representation of the same character). Human review required before deciding rollback.

RB-J2: T-series measurement at tolerance boundary. A timing measurement lands within 5ms of the tolerance boundary. A single such measurement does not trigger automatic rollback, but three consecutive boundary measurements require investigation and human decision.

Dimension 12: Required Human Review Checkpoints

The following extraction candidates require explicit human engineer review of the extraction diff before the change is applied. This is not optional. Skipping human review for any of these candidates constitutes a governance violation.

H-01: Clinical calculation logic (CANDIDATE-015, 016, 017, 018)

Any extraction involving scoring weights, recommendation generation, risk classification, or treatment selection requires human clinical engineering review. The reviewer must confirm: (1) the extracted function's output is identical to the Tier 0 baseline for all Category A, B,

and C inputs, (2) the function's role in the calcAI vs. ClinicalEngine divergence is understood and preserved, (3) no clinical logic has been silently unified or separated that was previously divergent.

H-02: DEFAULT_STATE clinical field extraction (CANDIDATE-007)

Human review must confirm: (1) `editing`, `whyOpen`, and `historyOpen` are NOT present in the extracted module, (2) these three fields remain in the main application file's `DEFAULT_STATE`, (3) the `saveState()` destructure-blocklist pattern `const { editing, whyOpen, historyOpen, ...serializable } = s` is unchanged.

H-03: StateValidator extraction (CANDIDATE-021)

Human review must confirm: (1) the extracted validator has no side-channel write (no `StateDiffs` push, no event emission, no module-scoped array append), (2) the validator runs identically in isolation as it does inside `setState()`, (3) the extraction has not altered the contract between `setState()` and its validation behavior.

H-04: showToast() extraction (CANDIDATE-022, if approved)

Human review must confirm: (1) `showToast()` has no implicit dependency on `s`, (2) if it appends to `document.body`, the team has explicitly approved this as acceptable despite the DOM API scan rule, (3) no `setTimeout` for auto-dismiss is present.

H-05: Multi-call-site extractions

Any extraction requiring modification of more than one call site in the main application file requires human review to verify all call sites are equivalent replacements with identical argument passing.

H-06: Extractions from within `withErrorBoundary` closures

Any candidate that was previously called from inside a `withErrorBoundary` closure requires human review to confirm the error recovery behavior and the terminal error state contract are unaffected.

H-07: Post-rollback re-extraction

After any rollback event, human review is required before re-attempting the failed candidate. The failure reason must be documented and the revised extraction plan must be approved.

Dimension 13: Required Claude Opus Escalation Triggers

The following conditions require escalation to Claude Opus (full-context analysis) before extraction proceeds.

E-01: Incomplete coupling trace

A static analysis scan reveals a potential coupling path (S reference at depth-3 or greater, or an indirect call chain that cannot be fully traced within the current analysis context) for any candidate in CANDIDATE-007, CANDIDATE-019, CANDIDATE-020, CANDIDATE-022.

E-02: ClinicalEngine sub-function boundary ambiguity

When extracting any Tier 4 candidate (CANDIDATE-015, 016, 017, 018), the boundary between the pure sub-function and the routing logic (MISSING_SINGLE, MISSING_MULTI, RESTORATIVE branch) is ambiguous. Applies to CANDIDATE-016 and CANDIDATE-018 as the highest-risk candidates in this category.

E-03: calcAI purity ambiguity

`calcAI()` or `calcAIMulti()` appears to share a sub-function with a Tier 4 candidate, but the semantic difference between `calcAI()` and `ClinicalEngine.process()` for restorative conditions cannot be fully characterized within the current session. Applies to CANDIDATE-018 if the recommendation mapping function is confirmed to be called from the `calcAI()` path.

E-04: StateValidator side-channel discovered

`StateValidator.validate()` is found to push to a log, emit an event, or write to any data structure beyond its return value.

E-05: Unexplained timing regression

A T-series timing case falls outside tolerance after extraction and the cause cannot be identified from the extraction changeset within the current session.

E-06: Shared closure discovered between candidates

Two or more extraction candidates that appeared independent are found to share a closure over a module-scoped variable not caught by initial static analysis.

E-07: New orchestration dependency revealed

Evidence of an orchestration dependency not documented in the prior analyses is found

during extraction. This dependency must be fully characterized before extraction continues.

Dimension 14: Required Orchestration Verification

After every extraction in Tier 3 and Tier 4, the following orchestration sequences must be manually verified to execute in exactly the order documented.

render(S) orchestration sequence verification

Execute regression case R01. Using browser developer tools (performance profiler or console instrumentation), confirm the following call sequence occurs in this exact order with no inserted steps:

1. `render(S)` invoked
2. Branch evaluation: `S.multiSite && S.site2Tooth` → compound or default branch
3. `ClinicalEngine.process(S)` or `ClinicalEngine.processCompound(S)` invoked
4. `renderMainPanels(state, ai)` invoked with the processed result
5. All panel renders execute within `renderMainPanels()` (including
`buildAICardStructure()` / `updateAICard()` , `renderComparisonTable()` /
`lazyRenderComparisonTable()` , `renderMaterial()` , `renderCost()` , `renderRisk()` ,
`renderReasons()` , `renderPatientDisplay()` , `updateToothHighlight()` — Pass 1)
6. `updateToothHighlight(primaryTooth)` invoked directly from `render()` — Pass 2 (in compound mode only)
7. `updateToothHighlight2(secondaryTooth)` invoked directly from `render()` — Pass 2 (in compound mode only)

Any step appearing in a different position, any step missing, or any new step inserted between existing steps is an orchestration regression.

switchPatient() orchestration sequence verification

Execute regression case R06. Confirm the following sequence:

1. `Object.assign(S, newPatient)` executes synchronously
2. `showSkeleton('aiCardBody')` executes synchronously (clears `dataset.built`)
3. `History.render()` executes synchronously for new patient ID

4. `renderPatientList()` executes synchronously
5. `buildEditForm()` executes synchronously with new patient data
6. `setTimeout(() => { render(S); }, 300)` is registered — does NOT execute yet
7. 300ms elapses
8. `render(S)` executes for new patient

Confirm that between steps 6 and 8, the DOM shows: `#patientList` with new patient highlighted, `#historyList` with new patient history, `#aiCardBody` with skeleton markup, main panels still reflecting old patient data (or skeleton). This mixed DOM state is the documented existing behavior and must be preserved.

Dimension 15: Required Async Verification

Timer-active render verification

After every Tier 4 extraction, verify that a `render(S)` call issued while `_typewriterTimer` is mid-interval behaves identically to the Tier 0 baseline:

1. The new `render(S)` call must cancel the in-progress typewriter animation at the top of the next `typewriterEffect()` call (via `clearInterval(_typewriterTimer)`).
2. A new typewriter animation must start from empty.
3. The destroyed `recText` element content from the prior mid-animation state must not persist.

Any deviation from this sequence is an async cancellation regression.

Material timer cancellation verification

After every Tier 4 extraction, verify that rapid render calls cancel the prior `_matFadeTimer` correctly:

1. Trigger two `render(S)` calls within 160ms of each other.
2. Confirm that `#matPrimary` updates only once, at 160ms after the second render call.
3. Confirm that `#matPrimary` does NOT update at 160ms after the first render call.

This verifies the cancellation-via-overwrite pattern is intact.

Notes autosave silent abort verification

After every Tier 3 extraction and after any extraction adjacent to `_notePatient` capture, verify regression case R09:

1. Open a patient. Type in the notes textarea. Do not wait 800ms.
2. Switch to a different patient within 800ms of the last keystroke.
3. Confirm that the 800ms `_notesDebounce` fires but does NOT save the note content to the new patient.
4. Inspect `localStorage` to confirm the new patient's `notes` field is unchanged.

This is the notes autosave silent abort contract (Rule 11.7). Any extraction that disturbs `_notePatient` capture will cause the note content to be saved to the wrong patient.

Dimension 16: Required Stale Closure Verification

Comparison table observer stale closure

After every Tier 4 extraction, execute regression case R11:

1. Render a patient with a specific state.
2. Scroll `#comparisonTableBody` offscreen. The lazy render path sets `_compTableObserver` with `state` and `ai` captured by closure.
3. Trigger a state change (e.g., switch treatment). A new `render(S)` call disconnects the prior observer and creates a new one with the new state captured.
4. Without triggering a new render, scroll `#comparisonTableBody` back into view.
5. Confirm that the comparison table renders with the state captured at observer creation time (the state at the time of the most recent `render(S)` call), not with a hypothetical "more current" state.

This is not a bug — it is the documented existing stale closure behavior (Rule 11.5, Boundary B-07). After extraction, this behavior must be identical to the Tier 0 baseline. Any extraction that introduces a mechanism that refreshes the observer's captured values is an observer integrity regression.

Preview mode stale banner

After every extraction, execute regression case R06 in the variant where the prior patient

was in preview mode:

- 1. Open a patient. Activate preview mode (move a simulator slider).
- 2. Switch to a different patient without applying the preview.
- 3. Confirm that `previewModeBanner.style.display` is still `'flex'` on the new patient's view.
- 4. Confirm that `_isPreviewMode` is still `true`.

This is the documented stale banner behavior (DOM Lifecycle Audit, Section 5). No extraction may introduce a banner reset on patient switch.

Dimension 17: Required Timer Integrity Verification

Timer registry invariant

After every extraction, the timer registry must contain exactly the same timer types as the Tier 0 baseline. The following timers must be present and registered in the main application file (not in any extracted module):

Timer	Type	Duration	Registry
<code>_typewriterTimer</code>	<code>setInterval</code>	30ms	None (next invocation clears)
<code>_matFadeTimer</code>	<code>setTimeout</code>	160ms	None (next invocation clears)
Anonymous report restore	<code>setTimeout</code>	800ms	None (orphaned)
CleanupRegistry URL revoke	<code>setTimeout</code>	1200ms	CleanupRegistry
<code>_saveStateTimer</code>	<code>setTimeout</code>	200ms	None (next invocation clears)
<code>_notesDebounce</code>	<code>setTimeout</code>	800ms	None (next invocation clears)
Anonymous patient switch render	<code>setTimeout</code>	300ms	None (orphaned per switch)
CleanupRegistry slider preview	<code>setTimeout</code>	120ms	CleanupRegistry

Anonymous save indicator hide	setTimeout	2000ms	None (orphaned)
-------------------------------	------------	--------	-----------------

After every extraction, verify:

- No timer from this list has been removed from the main application file.
- No timer from this list has been moved into an extracted module.
- No new timer has been added anywhere in the codebase.
- `_matFadeTimer` is not in `CleanupRegistry` (it survives pagehide cleanup — this is existing behavior that must be preserved, not fixed).

CleanupRegistry integrity

The `CleanupRegistry` must contain exactly the same registered timers and observers after extraction as before. The 1200ms URL revoke timer and the 120ms slider preview timer must remain registered. `_matFadeTimer`, `_typewriterTimer`, and the 800ms report button restore timer must remain unregistered. This asymmetric registry state is a documented existing behavior.

Dimension 18: Required Observer Integrity Verification

`_compTableObserver` lifecycle

After every extraction, verify:

- `_compTableObserver` is a module-scoped variable in the main application file. It must not be moved into any extracted module.
- `_compTableObserver.disconnect()` is called at the top of every `lazyRenderComparisonTable()` invocation before a new observer is created.
- The new observer captures `state` and `ai` by closure at creation time. The closure captures are not refreshed by any mechanism.
- `updatePreview()` does NOT use `lazyRenderComparisonTable()` — it calls `renderComparisonTable()` directly, bypassing the observer entirely. This bypass must remain intact.
- `toggleMultiSite()` does NOT interact with `_compTableObserver` — it writes directly to `_tbody.innerHTML`. This direct write must remain intact.

Observer creation timing

After any Tier 4 extraction, verify that `_compTableObserver` is created (or disconnected and re-created) at the same point in the render cycle as in the Tier 0 baseline. An observer created earlier or later than expected may produce a different stale closure capture, altering comparison table content at scroll-into-view time.

Dimension 19: Required Render-Order Verification

Two-pass SVG compound highlight order

After every Tier 4 extraction, execute regression case R05 (multi-site compound mode). Using DOM mutation observers or console instrumentation, record the sequence of `updateToothHighlight()` and `updateToothHighlight2()` calls and confirm:

1. `updateToothHighlight(effectiveState.tooth)` fires from within `renderMainPanels()` — this is Pass 1.
2. `updateToothHighlight(primaryTooth)` fires from within `render()` — this is Pass 2, and it overwrites Pass 1.
3. `updateToothHighlight2(secondaryTooth)` fires from within `render()` immediately after Pass 2.
4. The final SVG attribute state reflects Pass 2 values, not Pass 1 values.

If Pass 1 values survive as the final SVG state, or if Pass 2 does not execute, or if the order is reversed, this is a two-pass SVG regression — the most severe orchestration regression possible in Wave 1, as it directly affects clinical display output.

renderTxCards → updateCrownCardState ordering

After every Tier 4 extraction, verify that `renderTxCards()` executes before `updateCrownCardState()` within the same render cycle. `updateCrownCardState()` reads `.classList` state written by `renderTxCards()`. If this ordering is disturbed, `updateCrownCardState()` reads stale or absent class state, producing incorrect treatment card display.

buildAICardStructure → updateAICard ordering

Verify that `buildAICardStructure()` always executes before `updateAICard()` or `updateAICardMulti()` within a given render cycle. `updateAICard()` patches child elements whose IDs only exist if `buildAICardStructure()` has already executed. If `updateAICard()`

executes without a prior `buildAICardStructure()`, all `$()` selectors return null, producing a silent no-op render with missing AI card content.

Dimension 20: Required Semantic Equivalence Verification

calcAI vs. ClinicalEngine divergence preservation

This is the most critical semantic invariant in Wave 1. The documented behavioral divergence between `calcAI()` and `ClinicalEngine.process()` for restorative conditions is an existing condition that must not be altered, unified, or extended by any Wave 1 extraction.

After every Tier 4 extraction, run the following verification sequence:

1. Set patient state to a restorative condition case (crown-eligible, crown-viable — regression case R02).
2. Call `calcAI(S)` directly and record the full output object.
3. Call `ClinicalEngine.process(S)` directly and record the full output object.
4. Compare the two outputs. The differences between them must be byte-for-byte identical to the differences recorded in the Tier 0 baseline for the same input.

If the differences increase (the two paths now diverge more than they did before extraction), a Tier 4 extraction has inadvertently changed one path but not the other. Rollback immediately. This is RF-05.

If the differences decrease (the two paths now converge toward the same output), a Tier 4 extraction has inadvertently unified logic that was intentionally separate. Rollback immediately. This is also RF-05.

Any change in the differential — in either direction — is a semantic equivalence regression.

Recommendation label equivalence

After every Tier 1 extraction of recommendation label maps (CANDIDATE-008) and every Tier 4 extraction of recommendation mapping functions (CANDIDATE-018), verify that recommendation labels produced by the AI card for regression cases R01, R02, R04, and R05 are byte-for-byte identical to the Tier 0 baseline. Label changes that appear cosmetically identical (e.g., different Unicode quotation marks, different whitespace normalization) are still regressions.

Condition category classification equivalence

After every Tier 1 extraction of condition category maps (CANDIDATE-006) and every Tier 4 extraction using those maps as dependencies, verify that condition-to-category classification produces identical results for all conditions present in the regression test patient set.

Score normalization equivalence

After CANDIDATE-015 (score normalization) extraction, verify that normalized scores for all regression cases are numerically identical to the Tier 0 baseline to full floating-point precision. Rounding differences, floating-point precision differences, or any numeric deviation constitute a semantic regression.

Validation Checklist Template

The following checklist must be completed and documented for every extraction before the next extraction begins. All entries require a binary PASS/FAIL result. Any single FAIL halts the extraction sequence.

WAVE 1 EXTRACTION VALIDATION RECORD

Extraction target:

Extraction tier:

Rollback checkpoint before apply:

Rollback checkpoint after apply:

DIMENSION 1 – VALIDATION SCOPE

- [] Isolation pass (pure Node.js, no mocks required)
- [] All call sites reference imported version
- [] No duplicate inline definition remains
- [] R-series scope for tier verified (list cases verified):

DIMENSION 2 – BEHAVIORAL BASELINES

- [] Category A baseline recorded
- [] Category B baseline recorded
- [] Category C baseline recorded (if applicable)
- [] Post-extraction output byte-for-byte identical to baseline

DIMENSION 3 – RENDER INVARIANTS

- [] AI card structural/patch distinction unchanged
- [] dataset.built flag behavior unchanged
- [] withErrorBoundary terminal state unchanged

- [] Comparison table full-replacement behavior unchanged
- [] _compTableObserver stale closure behavior unchanged
- [] Material 160ms deferred write unchanged
- [] Two-pass SVG sequence unchanged (Tier 4 only)
- [] Patient list full-replacement behavior unchanged

DIMENSION 4 – SYNCHRONIZATION INVARIANTS

- [] render(S) orchestration sequence unchanged
- [] switchPatient() 300ms window unchanged
- [] saveState → renderPatientList chain unchanged
- [] Preview vs. apply race condition unchanged
- [] whatIfApply promotion field set unchanged

DIMENSION 5 – PERSISTENCE INVARIANTS

- [] Transient flag blocklist (editing, whyOpen, historyOpen) intact
- [] Patient records in localStorage unchanged
- [] History entry patient association unchanged
- [] Active patient ID unchanged

DIMENSION 6 – TIMING INVARIANTS (Tier 3 and 4 only)

- [] T01: #matPrimary update at 160ms ± 20ms (3 measurements)
- [] T02: Patient switch full update at 300ms ± 30ms
- [] T03: Notes autosave at 800ms ± 50ms
- [] T04: State autosave at 200ms ± 20ms
- [] T05: Typewriter interval at 30ms ± 5ms

DIMENSION 7 – DOM INVARIANTS

- [] innerHTML replacement map unchanged
- [] DOM element existence and content unchanged for R-series cases
- [] dataset.built flag state correct after render cycles
- [] SVG attributes correct for single-site and compound modes
- [] Preview banner state correct

DIMENSION 8 – localStorage INVARIANTS

- [] No transient flags in serialized records
- [] No cross-patient contamination
- [] History entries keyed to correct patient IDs

DIMENSION 9 – IMPORT/EXPORT VALIDATION

- [] Named exports only, no default exports
- [] No new imports beyond Wave 1 module import
- [] No barrel file created
- [] All call sites reference imported version

DIMENSION 10 – DEPENDENCY VALIDATION

- [] Full import chain resolves without main application file
- [] Depth-2 dependency scan: all dependencies pass purity scans

- [] No circular dependency at any depth
- [] No new npm packages added

DIMENSION 11 – ROLLBACK TRIGGERS

- [] None of RB-01 through RB-12 triggered

DIMENSION 12 – HUMAN REVIEW (if applicable)

- [] H-series condition applicable (Y/N, which):
- [] Human reviewer sign-off obtained

DIMENSION 13 – CLAUDE OPUS ESCALATION (if applicable)

- [] E-series condition triggered (Y/N, which):
- [] Escalation completed and documented

DIMENSION 14 – ORCHESTRATION VERIFICATION (Tier 3 and 4 only)

- [] render(S) call sequence matches documented order
- [] switchPatient() sequence matches documented order

DIMENSION 15 – ASYNC VERIFICATION (Tier 3 and 4 only)

- [] Timer-active render: typewriter cancellation correct
- [] Material timer cancellation: only second render fires
- [] Notes autosave silent abort: R09 passes

DIMENSION 16 – STALE CLOSURE VERIFICATION (Tier 3 and 4 only)

- [] Comparison table observer stale closure: R11 passes
- [] Preview mode stale banner: stale behavior preserved

DIMENSION 17 – TIMER INTEGRITY VERIFICATION (Tier 3 and 4 only)

- [] Timer registry unchanged
- [] No timer moved into extracted module
- [] No new timer introduced
- [] CleanupRegistry unchanged

DIMENSION 18 – OBSERVER INTEGRITY VERIFICATION (Tier 3 and 4 only)

- [] _compTableObserver remains in main application file
- [] Observer disconnect-then-recreate lifecycle intact
- [] updatePreview() bypasses observer (direct renderComparisonTable())
- [] Observer creation timing matches Tier 0 baseline

DIMENSION 19 – RENDER-ORDER VERIFICATION (Tier 4 only)

- [] Two-pass SVG: Pass 2 is authoritative final state
- [] renderTxCards precedes updateCrownCardState in render cycle
- [] buildAICardStructure precedes updateAICard in render cycle

DIMENSION 20 – SEMANTIC EQUIVALENCE VERIFICATION (Tier 4 only)

- [] calcAI vs. ClinicalEngine divergence: differential unchanged
- [] Recommendation label output: byte-for-byte identical to baseline

```
[ ] Condition category classification: identical to baseline
[ ] Score normalization output: numerically identical to baseline

FINAL STATUS: [ ] PASS — proceed to next extraction
               [ ] FAIL — rollback to wave1-ckpt-{tier}-{n}
Failure mode documented:
Re-extraction plan approved by human:
```

Behavioral Drift Definition

Behavioral drift is any change in observable application output that was not present in the Tier 0 baseline, regardless of whether the change appears intentional, cosmetic, or inconsequential. The following categories constitute behavioral drift for Wave 1 purposes:

Output drift: Any change in formatted strings, computed values, labels, classification results, or score outputs returned by extracted functions.

Timing drift: Any measurable change in the intervals of `_matFadeTimer`, `_typewriterTimer`, `_saveStateTimer`, `_notesDebounce`, the 300ms patient switch defer, or any CleanupRegistry-managed timer.

DOM drift: Any change in which DOM elements are written to, when they are written to, by which function they are written to, or what content they receive.

Persistence drift: Any change in which fields appear in serialized `localStorage` records, including the addition of transient flags (`editing`, `whyOpen`, `historyOpen`) to persisted data.

Orchestration drift: Any change in the order of function calls within a render cycle, including the insertion or removal of any call, or the movement of any call to a different point in the sequence.

Semantic drift: Any change in the differential between `calcAI()` output and `ClinicalEngine.process()` output for the same input state, in either direction.

Closure drift: Any change in which values are captured by `_compTableObserver` at creation time, or any introduction of a mechanism that refreshes those captures.

All categories of drift require immediate rollback. There is no acceptable drift in Wave 1.

Document status: Deterministic extraction validation governance. Binding for all Wave 1

extraction activity. No architectural changes, refactors, optimizations, or redesigns are contained herein or implied. All validation requirements are derived from documented system behavior in the synchronization analysis, render trigger chain analysis, architecture risk consolidation, DOM lifecycle audit, Wave 1 Execution Playbook, and Wave 1 Candidate Registry.